Final Report - TB Sol for Venus

Name of submission: Final technical report

Group name: Testbench for Bit-Serial RISC-V processor in SoI for Venus

Date: 2026/01/06

Organization: KTH

Authors:

Name	Email
Chinmay Purandare	chinmayp@kth.se
Lorenzo Parata	parata@ug.kth.se
Chongnan Wang	chongnan@kth.se
Xiaotian Jiang	xjia@kth.se
Xinxin Qin	xinxinq@kth.se
Paul Martin	paulmart@kth.se

Department of Electronics
KTH Royal Institute of Technology
Stockholm, Sweden

Abstract

This project implements and validates a dual-FPGA testbench system for testing RISC-V processor architectures under severe I/O constraints. The work completes the hardware verification phase of two processor designs previously developed at KTH: a conventional parallel RISC-V processor and a bit-serial CISC-V processor, both optimized for high-temperature environments.

The testbench architecture employs a master-slave configuration where one FPGA hosts the processor core while a second FPGA implements the complete test environment, including memory models, control logic, and verification infrastructure. Communication between the two FPGAs is achieved through a custom 2-bit serial protocol, limiting the inter-FPGA connection to only 10 pins out of the maximum 12 available. This stringent constraint reflects the physical limitations of KTH's high-temperature test station, which operates at temperatures approaching 480°C for Venus surface conditions.

The project successfully demonstrates both processor implementations operating correctly in hardware, with comprehensive functional verification through differential testing against architectural simulators. The bit-serial communication protocol, while introducing significant latency overhead, proves effective for reliable data transfer and enables complete instruction fetch, data memory access, and control operations within the pin budget constraints.

This work represents a critical validation step for high-temperature digital systems designed for extreme planetary environments, proving that complex processor architectures can be effectively tested and verified under severe I/O limitations imposed by specialized test equipment. The developed testbench infrastructure provides a reusable platform for future processor designs targeting similar environmental constraints.

Contents

Abstract 1

Acronyms 4

1 Introduction 7

2 Synthesis 9
- 2.1 RiscV . . . 9
  - 2.1.1 Introduction . . . 9
  - 2.1.2 ISA . . . 10
  - 2.1.3 RISCV So Architecture . . . 11
- 2.2 Bit-Serial CISC-V . . . 12
- 2.3 Synthesis Overview . . . 13
  - 2.3.1 RISC-V Synthesis . . . 13
  - 2.3.2 Brisc-V Synthesis . . . 14
- 2.4 CPU Timing Analysis . . . 15

3 Processor 16
- 3.1 System Architecture Overview . . . 16
  - 3.1.1 Top-Level Entity: CiscV_top . . . 16
  - 3.1.2 Reset Synchronization . . . 17
  - 3.1.3 GPIO Pin Mapping . . . 17
  - 3.1.4 Debug LED Assignment . . . 18
- 3.2 Clock Management . . . 18
  - 3.2.1 Clock Divider Implementation . . . 18
- 3.3 CISC-V Processor Core . . . 19
  - 3.3.1 ProGram-Based FSM Architecture . . . 19
  - 3.3.2 Internal Signal Structure . . . 19
  - 3.3.3 Register File and Arithmetic Logic . . . 20
  - 3.3.4 Program Counter Management . . . 21
  - 3.3.5 Bit-Serial Communication Protocol . . . 21
  - 3.3.6 Memory Interface Timing . . . 22
- 3.4 Pin Constraints and Timing Analysis . . . 22
  - 3.4.1 Pin Assignment Configuration . . . 22
  - 3.4.2 Timing Constraints . . . 23

4 Testbench 24
- 4.1 Initial Simulation Testbench . . . 24
  - 4.1.1 Testbench Architecture . . . 24
  - 4.1.2 Toolchain and Environment Debugging . . . 25
- 4.2 Differential Verification Testbench . . . 26
  - 4.2.1 Differential Verification Setup and Trace Alignment . . . 26

4.2.2 Mismatch Symptoms and Root Cause Localization 27
4.2.3 Fix for Sub Word Loads and Trace Consistency 28
4.3 Synthesizable Testbench 28
4.3.1 Synthesizable Platform Refactoring and IP Based Memories 29
4.3.2 Seven Segment Instruction Count Display 30
4.4 Duel-board Testbench 30
4.4.1 Interface Transform Module 31
4.4.2 IO Design 32
4.4.3 Dual Board Bring Up 32

5 FPGA Communication **33**
5.1 TB Serialization - Bit Serial Interface 33
5.1.1 bs_interface 34
5.1.2 avm_interface 36

6 Conclusion **37**
6.1 Conclusion 37
6.2 Future Work 37

7 References **38**

Acronyms

- **RISC-V** Reduced Instruction Set Computer - Five
- **SoI** Silicon-on-Insulator
- **TB** Testbench
- **CPU** Central Processing Unit
- **RAM** Random Access Memory
- **FPGA** Field-Programmable Gate Array
- **I/O** Input/Output
- **Brisc** Bit-Serial RISC-V (Processor Name)
- **FSM** Finite State Machine
- **Avalon** Avalon Memory-Mapped Interface

List of Figures

1 RV32I instruction formats 10
2 RV32I instruction formats 12
3 bit-serial CISC-V demonstration 13
4 Complete system architecture of FPGA 1 hosting the CISC-V processor 16
5 bit-serial interface architecture v1 34
6 bit-serial interface architecture v2 34
7 serialisation FSM graph 36

List of Tables

1 Timing analysis results for the RISC processor 15

1. Introduction

Modern space exploration missions demand electronic systems capable of withstanding extreme environmental conditions that render conventional technologies inoperative. Venus, with surface temperatures reaching 480°C and intense radiation exposure, represents one of the most challenging targets for electronic instrumentation. These extreme conditions necessitate fundamentally different approaches to digital system design and verification.

This project addresses the critical gap between processor design and hardware validation for extreme-environment applications. Building upon previous work at KTH, where two RISC-V processor variants were designed and synthesized—a parallel VHDL implementation and a bit-serial CISC-V model generated using ProGram—this work completes the verification chain by implementing both processors on FPGA hardware and developing a comprehensive dual-FPGA testbench system.

The architectural approach divides the system across two FPGAs: one hosting the processor core under test, and a second implementing the complete test environment including instruction memory, data memory, control logic, and verification infrastructure. This separation enables realistic testing conditions while accommodating the severe I/O constraints imposed by KTH's high-temperature test station, which limits inter-chip communication to a maximum of 12 physical probe connections. These constraints reflect the practical limitations of high-temperature test equipment, where conventional probe cards and connector technologies fail at elevated temperatures.

To operate within the 12-pad limitation, the system employs a custom bit-serial communication protocol that serializes all processor transactions over a compact 2-bit data interface. While this approach significantly increases transaction latency compared to parallel bus architectures, it successfully reduces the required pin count to 10 signals (including clock and control), enabling complete processor functionality within the constraint envelope. The protocol design encompasses instruction fetch, data memory access, and control signaling, with integrated parity checking for transaction integrity.

The project execution followed five main phases:

- **Toolchain and Hardware Setup**: Establishing the development environment, including Quartus Prime for FPGA synthesis, ModelSim for simulation, and configuration of DE0-Nano-SoC development boards.
- **FPGA Prototyping of Processor Cores**: Synthesizing both processor variants, resolving missing infrastructure components (particularly memory interfaces for the CISC-V design), and performing timing analysis to validate feasibility.
- **Testbench Development**: Implementing a progressively refined testbench architecture, from initial simulation-only environments through synthesizable hardware implementations, including memory models, execution control, and debug visibility features.

- **System Integration and Verification**: Physically connecting the two FPGAs, implementing the bit-serial protocol on both sides, establishing clock distribution, and conducting end-to-end functional testing with real benchmark programs.
- **Validation and Documentation**: Performing differential verification against architectural simulators, characterizing system performance, and documenting the complete design for future extension.

This report is structured to reflect the technical development progression. Section 2 presents the synthesis and timing analysis of both processor architectures, establishing their correctness and feasibility for FPGA implementation. Section 3 describes the processor implementation on FPGA 1, detailing the system architecture, clock management, reset synchronization, and the processor's internal organization including the ProGram-generated finite state machine. Section 4 documents the testbench development on FPGA 2, covering the evolution from simulation environments through synthesizable hardware implementations, including memory subsystems and execution monitoring. Section 5 details the inter-FPGA communication infrastructure, explaining the bit-serial protocol design, serialization mechanisms, and clock distribution strategy. Finally, Section 6 concludes with achieved results, lessons learned, and recommendations for future work extending this validation platform.

2. Synthesis

2.1. RiscV

2.1.1 Introduction

RISC-V is a novel instruction set architecture (ISA) originally designed to support computer architecture research and education, with the broader ambition of becoming a standard, free, and open architecture for industrial implementation. Unlike traditional commercial ISAs, which are proprietary and often restricted by intellectual property guardrails, RISC-V is completely open and freely available to both academia and industry. The primary design goals emphasize creating a real ISA suitable for direct native hardware implementation rather than just simulation, while avoiding "over-architecting" for specific microarchitectural styles or implementation technologies. This approach ensures efficiency across various designs, from ASICs to FPGAs, and supports highly parallel multicore or heterogeneous multiprocessor implementations.

The motivation for developing a new ISA stemmed from the significant limitations observed in existing commercial and open architectures. Dominant commercial ISAs are often characterized by complexity resulting from outdated design decisions and a lack of inherent extensibility, which leads to convoluted instruction encodings as new features are added. Furthermore, the market segmentation of these ISAs dilutes the benefits of their software ecosystems, as they are often not well-supported across different domains, such as server versus mobile space. While existing open ISAs like OpenRISC were considered, they were ultimately rejected due to technical constraints—such as the use of condition codes and branch delay slots—that complicate high-performance implementations and limit future expansion.

Structurally, RISC-V is defined as a modular architecture consisting of a mandatory base integer ISA and optional instruction-set extensions. The base integer ISA, available in 32-bit (RV32I) and 64-bit (RV64I) variants, is restricted to a minimal set of instructions sufficient to support compilers and operating systems, serving as a skeleton around which customized processors can be built. To facilitate general-purpose software development, the architecture defines standard extensions, including "M" for integer multiplication and division, "A" for atomic memory operations, and "F" and "D" for single- and double-precision floating-point arithmetic . The combination of the base integer ISA and these four standard extensions is designated as "G" (General).

Regarding instruction encoding and memory organization, the base RISC-V ISA employs fixed-length 32-bit instructions that must be naturally aligned on 32-bit boundaries. However, the encoding scheme is flexible enough to support optional variable-length instructions, where instructions can be composed of multiple 16-bit parcels. This capability enables the standard compressed ISA extension ("C"), which reduces code size by offering 16-bit compressed versions of common instructions and relaxing alignment constraints. The architecture follows a load-store

model, where arithmetic instructions operate only on CPU registers, and it utilizes a little-endian memory system by default.

2.1.2 ISA

The RISC-V instruction set architecture (ISA) is fundamentally structured around a mandatory Base Integer ISA, which must be present in any implementation, supplemented by a variety of optional extensions. While conceptually similar to early RISC processors, the base ISA is modernized by excluding branch delay slots and supporting optional variable-length instruction encodings. Designed as a minimal "skeleton," it is strictly restricted to the essential instructions required to support compilers, assemblers, linkers, and operating systems, thereby serving as a streamlined foundation upon which more complex, customized processors can be built.

Depending on the width of the integer registers and the size of the user address space, the base ISA is primarily categorized into RV32I (32-bit) and RV64I (64-bit) variants, alongside the RV32E subset designed for small microcontrollers and the future 128-bit RV128I variant. To facilitate general-purpose software development, RISC-V defines a set of standard extensions: "M" for integer multiplication and division, "A" for atomic memory operations used in inter-processor synchronization, and "F" and "D" for single- and double-precision floating-point arithmetic, respectively. The combination of the base integer ISA ("I") with these four standard extensions ("MAFD") is designated as "G" (General), which currently serves as the default target for compiler toolchains.

A distinguishing feature of the RISC-V architecture is its approach to stability. Unlike traditional architectures that typically update the entire ISA version as new instructions are added, RISC-V endeavors to keep the base ISA and standard extensions constant over time. New functionality is introduced by layering new optional extensions rather than redefining the base instructions. This design philosophy ensures that the base ISA remains a fully supported, standalone entity, allowing hardware designers to pursue deep specialization for energy efficiency or specific domains without compromising long-term software compatibility.

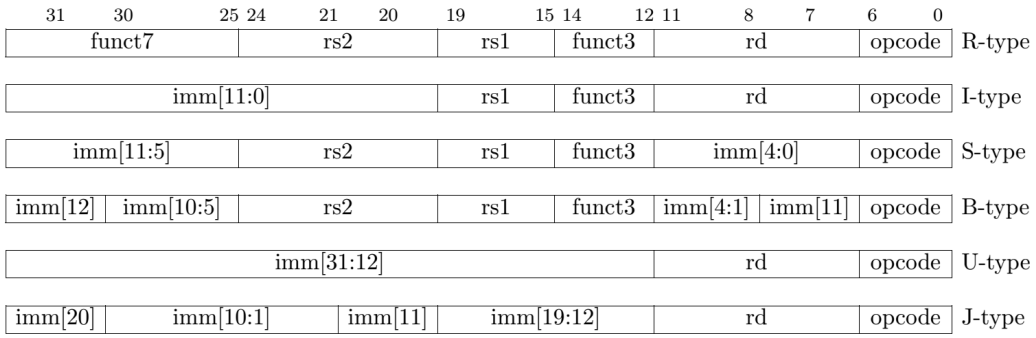

Figure 1: RV32I instruction formats

2.1.3 RISCV So Architecture

The proposed System-on-Chip (SoC) architecture is centered around a single RISC-V processing core, adhering to the modular and extensible design philosophy of the RISC-V instruction set. Fundamentally, RISC-V operates as a load-store architecture, meaning that arithmetic and logical operations are performed exclusively on CPU registers, while data transfer between memory and these registers is handled by dedicated load and store instructions. In RISC-V terminology, a "core" is defined by the presence of an independent instruction fetch unit. While the architecture supports hardware multithreading—allowing a single core to contain multiple hardware threads, or "harts"—our implementation adopts a single-hart configuration. This design choice prioritizes architectural simplicity and deterministic execution, avoiding the complexity of context-switching hardware in favor of a streamlined, single-threaded pipeline.

To address the limitations of the mandatory Base Integer ISA (e.g., RV32I), which serves as the minimal "skeleton" for control flow and simple integer manipulation, our design leverages the standard extension mechanism of the RISC-V ecosystem. Specifically, we incorporate a specialized hardware divider acting as a tightly coupled coprocessor. This unit implements the functionality of the RV32M (Multiply/Divide) standard extension, enabling the efficient execution of integer division and remainder instructions. By offloading these iterative and computationally intensive operations to fixed-function hardware, the system avoids the latency penalties associated with software-based division emulation, thereby significantly enhancing arithmetic throughput.

The integration of these computational elements is managed by a centralized system bus that connects the processor to its memory hierarchy and peripheral interfaces. The memory subsystem is partitioned into Read-Only Memory (ROM) for the secure storage of executable firmware and Random Access Memory (RAM) for dynamic runtime data. Interaction with the external physical world is facilitated by a General-Purpose Input/Output (GPIO) module. Consistent with the RISC-V architecture's approach to system-level programming, these I/O devices are memory-mapped. This allows the core to control peripheral states and read sensor data using the same standard load and store instructions employed for memory access, providing a unified and efficient addressing model for the entire SoC.

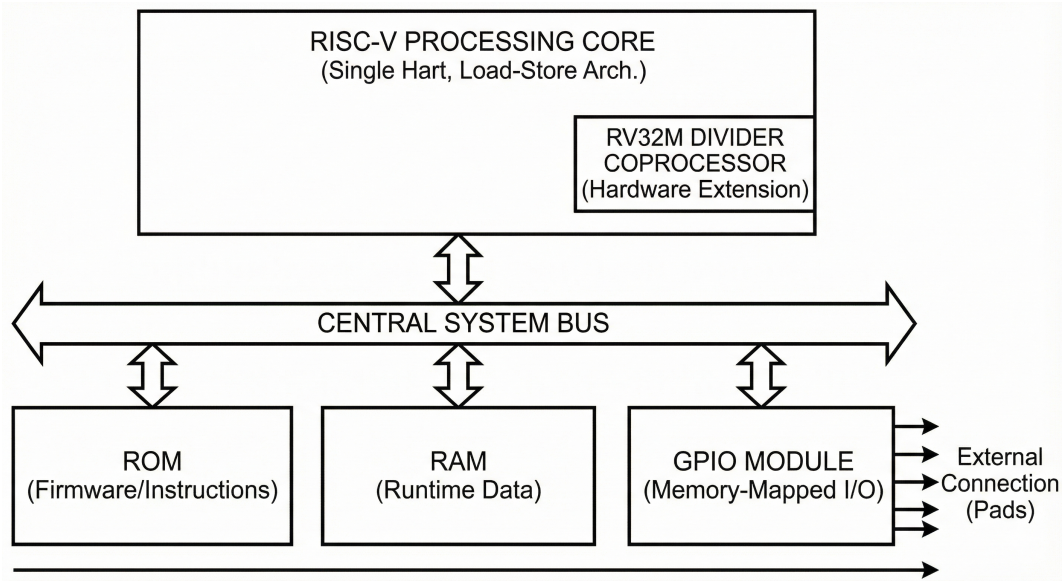

Figure 2: RV32I instruction formats

2.2. Bit-Serial CISC-V

The Bit-Serial CISC-V architecture is an experimental processor design explicitly engineered to overcome the severe physical constraints of high-temperature electronic testing environments. As the target high-temperature test station (utilizing Venus/Sol technology) is limited to only 12 physical probes (pads) due to the failure of standard measurement equipment at temperatures exceeding $400°$C, traditional parallel processor interfaces are rendered unfeasible. Consequently, this architecture adopts a bit-serial design paradigm intended to minimize I/O bandwidth requirements, thereby enabling reliable computation within an extremely restricted hardware interface.

The core innovation of this architecture lies in its unique execution mechanism, distinguishing it as a "CISC-V" variant. Unlike standard bit-parallel cache solutions, which typically require the processor to stall while waiting for a full data word to be assembled, the Bit-Serial CISC-V employs a non-stalling, streaming processing strategy. It is capable of executing instructions directly as the data bits are being loaded, effectively "processing-while-loading." This characteristic allows the architecture to maximize processing efficiency under the constraints of serial communication, mitigating the latency penalties often associated with traditional parallel-to-serial data conversion.

In terms of implementation, the model is described using the ProGram hardware description language, a choice that contrasts with the VHDL-based approach used for the project's parallel RISC-V model. To strictly adhere to the physical limitations of the test oven, the architecture features a specialized 2-bit serial interface, which successfully confines the design's footprint to the available 12 pads. Although the model has been successfully synthesized in previous research phases,

the primary objective of the current work is to prototype the design on FPGA hardware and perform timing verification to validate its operational integrity in simulated high-temperature environments.

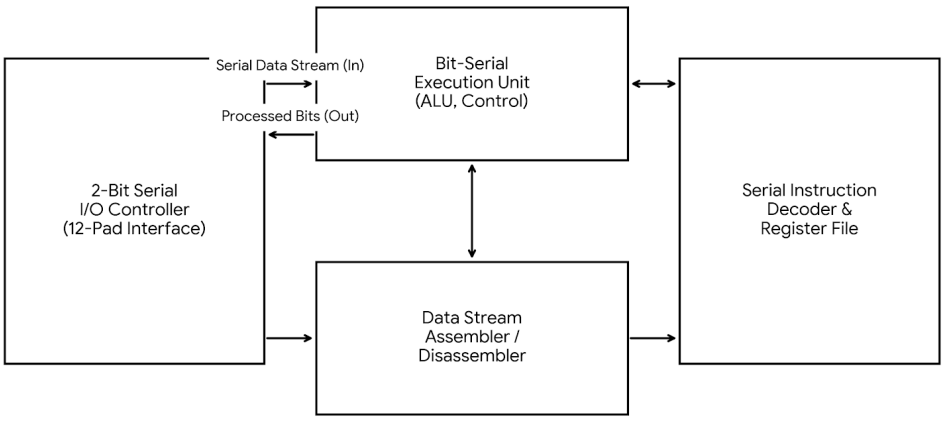

Figure 3: bit-serial CISC-V demonstration

2.3. Synthesis Overview

This section presents an overview of the synthesis and FPGA implementation process for both processor architectures we have in this project. As we previously seen, two main processor designs were synthesized and implemented: a conventional parallel RISC-V processor and the bit-serial Brisc-V processor. While the synthesis of the RISC-V core was relatively straightforward, the Brisc-V implementation required more work due to missing modules.

2.3.1 RISC-V Synthesis

The synthesis of the RISC-V processor did not present major difficulties. The design was successfully compiled, synthesized, and implemented on the FPGA using Intel Quartus without requiring structural modifications. The only challenge was to set Quartus parameters, import the right module and last but not least connect to the right FPGA board. Some of us had some difficulties in the matter, especially those who used a virtual machine.

After synthesis, the design was implemented and tested on the FPGA, confirming correct integration of the processor. This part of the project mainly served as a reference implementation and validation baseline before moving to a more complex task.

2.3.2 Brisc-V Synthesis

The synthesis and implementation of the Brisc-V processor required additional work. The Brisc-V architecture is a bit-serial processor with a low number of I/O pads, which makes it particularly interesting for our applications. However, the original design provided only the processor core and assumed the existence of external modules that were not included in the project deliverables.

During the initial synthesis attempts, two modules were missing: a memory module (RAM) and an I/O controller. These ones are required to allow the processor to access data memory, and interact with its environment. As a result, we add them and structure the Brisc-V system as follow:

- `Brisc_top`: Top-level entity of the system
- `CiscV`: Core of the bit-serial processor
- `bs_interface`: Bit-serial interface
- `io_controller`: Communication with pins
- `RAM`: Wrapper module interfacing the processor with on-chip memory
- `RAM_1PORT`: Single-port memory generated using the Quartus IP Catalog

RAM Module Implementation The decision was made to use Quartus IP Catalog for generating a single-port memory. It allowed us to rely on a proven and optimized memory implementation. The generated memory, `RAM_1PORT`, was configured with the following parameters:

- Single-port memory
- VHDL implementation
- Word width of 32 bits
- Memory depth of 2^{10} words (1024 words)
- Pre-filled memory content enabled

Those parameters, especially word width and memory depth, correspond to the constant of the package type_pkg that is in the repository project. That allowed a perfect fit.

However, the I/Os of the generated memory doesn't correspond to the I/Os of the component called by the top module. A wrapper needs to be implemented too. As a result, we wrote a `RAM` wrapper module that is responsible for connecting the processor core to the generated memory block. It adapts the address, data, and control signals of the Brisc-V core to the interface expected by the `RAM_1PORT` IP. This includes managing read and write accesses and ensuring correct word alignment.

Once the RAM module was implemented and integrated into the top-level design, the Brisc-V processor could be successfully synthesized and implemented on the FPGA using Quartus. A simple dummy instead of the RAM could have worked for the synthesis but would have no sense from a logic point of view. So this step was essential to enable further testing later.

2.4. CPU Timing Analysis

Now that we are sure both CPUs are synthesized and can be run on FPGA. We can perform a timing analysis of the different processor cores. As we said, all processors were synthesized using Quartus, and we will use the same tool for the analysis. However, the initial synthesis results did not include any timing analysis because Quartus was unable to properly identify and constrain the clock domain of the design.

The root cause of this issue was that the internal clock signal of the processor cores (`clk`) was not directly connected to the physical clock input of the FPGA board (`FPGA_CLK_50`). As a consequence, Quartus couldn't generate any timing report. To resolve this problem, a wrapper module was introduced and defined as the new top-level entity of the design. This wrapper explicitly connects the FPGA hardware signals, including the board clock, reset signal, and output peripherals such as LEDs, to the internal processor core. As a result, the processor clock is driven directly by the `FPGA_CLK_50` signal. In addition, a simple sequential process was added inside the wrapper to drive a blinking LED. This step was necessary so Quartus doesn't optimize away the entire design during synthesis. Indeed, without observable logic driven by the clock, the synthesis tool may consider parts of the design as redundant and decide to not use them.

After that, Quartus successfully generated timing analysis reports for the processor cores. However, the result showed negative setup slack values. It means that the design did not meet the default timing assumptions. This result is totally normal since no explicit timing constraints had been defined for the clock signal. So we wrote a Synopsys Design Constraints (SDC) file to properly constrain the clock period at 20ns. After applying the SDC constraints, the timing violations were resolved and the designs met the desired timing requirements. Table 1 summarizes the observed setup and hold slack values for the RISC processor before and after applying the SDC constraints.

	Setup Slack (ns)	Hold Slack (ns)
Without SDC	≈ -1	≈ 0.3
With SDC	≈ 11	≈ 0.3

Table 1: Timing analysis results for the RISC processor

As shown in the table, the application of clock constraints significantly improved the setup slack that is now positive, it confirms the validity of the processor.

Quartus also showed us several cases, especially the worst case scenario at min and max temperature, and in all cases the slack remained positive.

The same methodology was applied to the BRISC processor and it shows the same result. As a result, this part validates the feasibility of both processor designs under realistic clocking conditions (20ns).

3. Processor

3.1. System Architecture Overview

The first FPGA serves as the processor host platform, implementing the complete CISC-V bit-serial processor system along with the necessary infrastructure for communication with the testbench FPGA. The top-level architecture consists of several key modules that handle clock management, processor execution, and inter-FPGA communication.

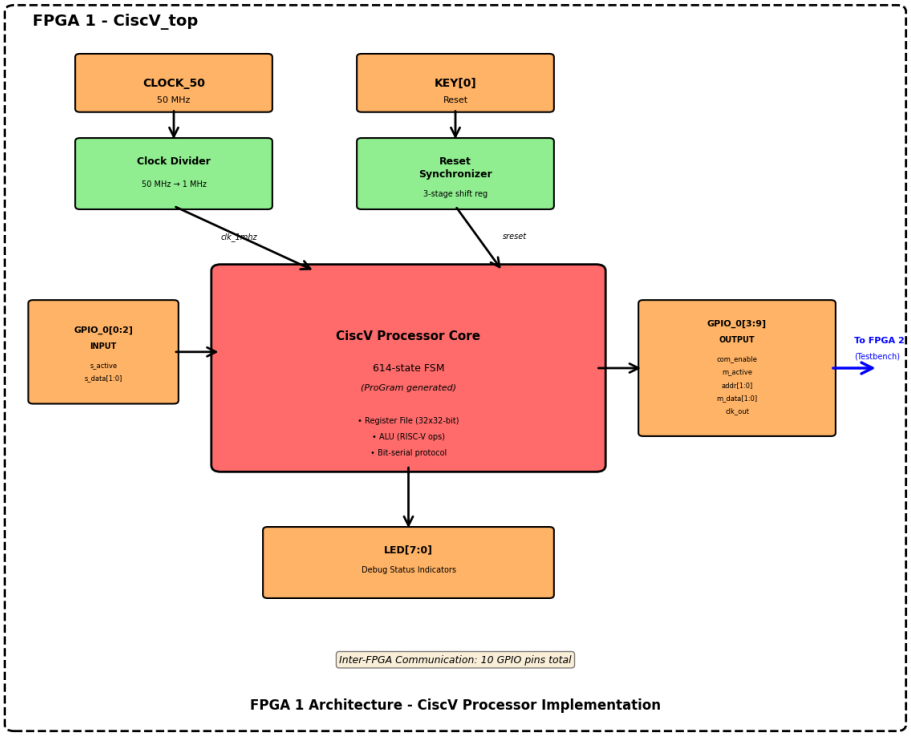

Figure 4: Complete system architecture of FPGA 1 hosting the CISC-V processor

3.1.1 Top-Level Entity: CiscV_top

The `CiscV_top` entity serves as the system's top-level wrapper, providing the interface between the FPGA's physical pins and the internal processor subsystem. The entity exposes three main groups of external signals:

- **Clock and Reset**: A 50 MHz clock input (`CLOCK_50`) and an active-low reset button (`KEY[0]`)
- **GPIO Interface**: A 10-pin bidirectional GPIO bus (`GPIO_0[9:0]`) for inter-FPGA communication
- **Debug LEDs**: Eight LEDs (`LED[7:0]`) for runtime status indication

The structural architecture instantiates two primary components: the `CiscV` processor core and a `clock_divider` module. The clock divider reduces the 50 MHz board clock to 1 MHz, which serves as the processor's operating frequency. This frequency reduction is necessary to ensure reliable operation of the bit-serial communication protocol and to meet timing constraints for the processor pipeline.

3.1.2 Reset Synchronization

A critical aspect of the design is the proper handling of asynchronous reset signals. The KEY button provides an asynchronous active-low reset, which must be synchronized to the system clock domain to prevent metastability issues. The implementation uses a three-stage shift register to synchronize the reset signal:

```
reset_sync_proc: PROCESS(CLOCK_50, reset_n)
   VARIABLE reset_shift : std_logic_vector(2 downto 0) := "000";
BEGIN
   IF reset_n = '0' THEN
      reset_shift := "000";
      reset_sync <= '1';
   ELSIF rising_edge(CLOCK_50) THEN
      reset_shift := reset_shift(1 downto 0) & '1';
      reset_sync <= NOT reset_shift(2);
   END IF;
END PROCESS;
```

This synchronizer ensures that the internal reset signal (`sreset_sig`) is properly aligned with the clock domain before being distributed to the processor core.

3.1.3 GPIO Pin Mapping

The GPIO pins are carefully mapped to support the bit-serial communication protocol. The mapping follows a master-slave convention where the processor acts as the master initiating transactions:

Input Signals (from testbench FPGA):

- `GPIO_0(0)`: Slave active acknowledgment (`s_active`)
- `GPIO_0(1:2)`: Received data bus (`s_data[1:0]`)

Output Signals (to testbench FPGA):

- `GPIO_0(3)`: Communication enable (`com_enable`)
- `GPIO_0(4)`: Master active flag (`m_active`)
- `GPIO_0(5:6)`: Address/data transmission bus (`addr[1:0]`)
- `GPIO_0(7:8)`: Transmitted data bus (`m_data[1:0]`)
- `GPIO_0(9)`: Forwarded clock signal (`clk_out`)

This pin assignment uses 10 of the available GPIO pins, staying well within the 12-pin constraint imposed by the high-temperature test environment requirements.

3.1.4 Debug LED Assignment

The LED outputs provide real-time visibility into the processor's communication state. Each LED is mapped to a specific internal signal, allowing hardware-level debugging without requiring external instrumentation:

- `LED[0]`: Communication enable status
- `LED[1]`: Master active flag
- `LED[2:3]`: Current address bits
- `LED[4:5]`: Current data bits being transmitted
- `LED[6]`: Clock output status
- `LED[7]`: Divided clock signal (1 MHz)

This mapping enables visual confirmation that the processor is actively communicating and provides immediate feedback during bring-up and debugging phases.

3.2. Clock Management

3.2.1 Clock Divider Implementation

The `clock_divider` module implements a frequency divider that generates the 1 MHz processor clock from the 50 MHz board clock. The division is achieved using a simple counter-based approach:

```
CONSTANT DIV_VALUE : integer := 25;
SIGNAL counter : integer range 0 to DIV_VALUE-1 := 0;
SIGNAL clk_reg : std_logic := '0';
```

The divider counts to 25, then toggles the output clock. Since the output toggles every 25 input cycles, the effective division ratio is 50, producing a 1 MHz output from the 50 MHz input. This implementation ensures a 50% duty cycle, which is important for reliable synchronous logic operation.

The counter-based approach was chosen over PLL-based clock generation for several reasons:

- Simplicity and deterministic behavior
- No phase-locked loop lock time delays during reset
- Predictable jitter characteristics
- Lower resource utilization

3.3. CISC-V Processor Core

3.3.1 ProGram-Based FSM Architecture

The `CiscV` entity represents the bit-serial RISC-V processor core, automatically generated using the ProGram hardware description language. ProGram is a high-level synthesis tool that generates optimized finite state machines from algorithmic descriptions. The resulting VHDL code implements a 614-state FSM that controls all aspects of processor operation, including instruction fetch, decode, execution, and bit-serial communication.

The FSM-based architecture is particularly well-suited for bit-serial operation because it naturally expresses the sequential nature of bit-by-bit data transmission and reception. Each state represents a specific phase of operation, and state transitions are triggered by both internal processor events and external communication handshakes.

3.3.2 Internal Signal Structure

The processor maintains an extensive set of internal signals that represent the architectural state and control information:

Instruction Decoding Signals:

- `rd_data[4:0]`: Destination register index
- `rs1_data[4:0]`: Source register 1 index
- `rs2_data[4:0]`: Source register 2 index
- `imm5_data`, `imm7_data`, `imm12_data`, `imm20_data`: Various immediate field formats

Execution Control Signals:

- `execution_enable`: Triggers ALU or branch execution
- `execution_type`: Distinguishes between ALU and branch operations
- `result_select[2:0]`: Multiplexer control for selecting result source

Memory and I/O Signals:

- `load_byte_enable`, `load_half_enable`: Enable sub-word load operations
- `store_enable`: Initiates memory write operations
- `load_result[31:0]`: Buffer for reconstructed load data

3.3.3 Register File and Arithmetic Logic

The processor includes a 32-entry register file implemented as an internal memory array:

```
type memory_array_type is array (natural range <>)
    of std_logic_vector(31 downto 0);
signal regs:memory_array_type(31 downto 0);
```

Register writeback is controlled by a dedicated process that monitors the `writeback_enable` signal and updates the destination register specified by `rd_data`. The writeback logic includes special handling for load instructions, which require an additional pipeline stage to reconstruct the full word from serially received data.

The ALU implementation supports the full set of RISC-V integer operations:

- **Arithmetic**: ADD, SUB
- **Logical**: AND, OR, XOR
- **Shift**: SLL (shift left logical), SRL (shift right logical), SRA (shift right arithmetic)
- **Comparison**: SLT (set less than), SLTU (set less than unsigned)

Each operation is implemented as a combinational assignment within a large case statement, with the operation code determined by the `alu_ctrl` signal decoded from the instruction.

3.3.4 Program Counter Management

The program counter (PC) update logic handles all control flow scenarios defined by the RISC-V ISA:

- **Sequential execution** (`pc_sel = "001"`): PC increments by 4
- **JAL/JALR jumps** (`pc_sel = "010"` or `"011"`): PC is set to the computed target address
- **Conditional branches** (`pc_sel = "100"`): PC is conditionally updated based on branch condition evaluation

The PC update is synchronized to the rising edge of the clock and respects the reset condition, initializing to address 0x00000000 when the system is reset.

3.3.5 Bit-Serial Communication Protocol

The processor implements a custom bit-serial protocol for instruction fetch and data memory access. The protocol operates over a 2-bit data bus, transmitting address and data information in a sequence of 2-bit symbols. Key aspects of the protocol implementation include:

Transaction Initiation:

- The processor asserts `m_active` to signal an active transaction
- The `com_enable` signal is asserted to enable communication
- Address bits are serialized and transmitted via the `addr[1:0]` bus

Data Transmission:

- Write data is serialized and transmitted through `m_data[1:0]`
- The transmission sequence reverses the bit order to match the testbench's expected format

Data Reception:

- The `receiving` signal indicates when the processor is expecting serialized input data
- Received data arrives through `s_data[1:0]` and is accumulated in internal FIFO structures

Parity Checking:

- The protocol includes parity computation for both address and data
- `parity_addr` and `parity_data` signals accumulate XOR-based parity
- The `reset_parity` signal reinitializes parity counters at transaction boundaries

This protocol design minimizes pin count while maintaining sufficient bandwidth for instruction fetch and data access operations, albeit at the cost of increased transaction latency compared to parallel bus implementations.

3.3.6 Memory Interface Timing

The memory access timing is tightly coupled to the bit-serial protocol. A complete 32-bit word transmission requires 16 clock cycles (2 bits per cycle $\times$ 16 cycles). Instruction fetches follow this pattern:

1. The processor transmits the instruction address (16 cycles)
2. The testbench FPGA responds with the instruction word (16 cycles)
3. The processor receives and reconstructs the instruction
4. Instruction decoding begins

Similarly, data memory accesses for loads and stores follow the same serialization pattern, with additional handshaking for write acknowledgment.

3.4. Pin Constraints and Timing Analysis

3.4.1 Pin Assignment Configuration

The Quartus project settings file (`CiscV_top.qsf`) contains explicit pin assignments for all external signals. Each GPIO pin is constrained to a specific physical location on the FPGA:

```
set_location_assignment PIN_V12 -to GPIO_0[0]
set_location_assignment PIN_AF7 -to GPIO_0[1]
...
```

These assignments ensure consistent placement across compilation runs and guarantee that the inter-FPGA connections can be physically realized on the target board. The I/O standard for all GPIO pins is set to "3.3-V LVTTL" to match the electrical characteristics of the DE0-Nano-SoC board.

3.4.2 Timing Constraints

The Synopsys Design Constraints (SDC) file (`CiscV_top.sdc`) defines the timing requirements for the design:

Base Clock Constraint:

```
create_clock -name CLOCK_50 -period 20.000 [get_ports CLOCK_50]
```

This declares the input clock as having a 20 ns period (50 MHz frequency). The constraint is essential for timing analysis and ensures that the synthesis and place-and-route tools optimize the design to meet this requirement.

Generated Clock Constraint:

```
create_generated_clock -name clk_1mhz \
   -source [get_pins {clk_div|counter[0]}] \
   -divide_by 50 \
   [get_pins {clk_div|clk_reg}]
```

This derived clock constraint informs the timing analyzer about the relationship between the input clock and the divided clock. The divide-by-50 relationship is explicitly declared, allowing the tool to properly analyze timing paths that cross between the two clock domains.

Input and Output Delays: The SDC file includes input delay constraints for the GPIO input signals and output delay constraints for the GPIO output signals. These constraints model the board-level timing between the two FPGAs:

```
set_input_delay -clock clk_1mhz -max 5.0 [get_ports {GPIO_0[0]}]
set_output_delay -clock clk_1mhz -max 5.0 [get_ports {GPIO_0[3]}]
```

The 5 ns delay constraints provide margin for board trace delays and setup/hold requirements of the receiving FPGA's input registers.

False Path Declarations:

```
set_false_path -from [get_ports {KEY[0]}] -to [all_registers]
set_false_path -to [get_ports LED[*]]
```

These declarations inform the timing analyzer that the reset path and LED outputs are asynchronous and should not be analyzed for timing violations. The reset synchronization logic handles metastability internally, and the LEDs are for debug purposes only and do not require tight timing.

4. Testbench

4.1. Initial Simulation Testbench

The first milestone was to build a minimal yet reliable simulation environment around the provided RISCV processor while treating it strictly as a black box. The goal was to avoid any dependency on internal microarchitectural details and to interact only through the processor top level behavior.

4.1.1 Testbench Architecture

The processor exposes four functional groups of connections. First, an instruction fetch connection where the processor provides an instruction address and a read request, and the environment responds with a 32-bit instruction word and an optional wait indicator. Second, a data access connection where the processor provides an address plus read or write intent, write data, and byte enables, and the environment responds with read data and an optional wait indicator. Third, a simple control connection used to start execution, where the environment issues a write of a specific start value to release the processor into normal operation. Fourth, a set of debug outputs used for observability, including the current program counter, the current instruction word, and register writeback information consisting of a write enable event, a destination register index, and the value being written.

A structural rule was enforced from the beginning to keep the work scalable and reusable. The top wrapper was kept intentionally simple and only instantiated the processor and the test environment, provided clock and reset, and wired signals between modules. All behavioral testbench logic was placed in a dedicated environment module. This separation ensured that the environment remained a clean and portable abstraction that could later be migrated toward hardware oriented test setups without entangling clock and reset generation or instantiation details.

A key design choice was to make the environment interface complementary to the processor interface. The processor drives addresses and read or write intents, while the environment drives read data and wait responses. This choice is important for reuse because it matches how a real platform would interact with a processor. It also prevents accidental testbench code that writes into signals that should be driven by the processor, which is a common source of both compile time and runtime failures. With this interface discipline in place, the baseline environment implemented two simple memory models. The instruction memory was modeled as a word addressed array that returns a 32 bit instruction corresponding to the requested address. The data memory was modeled as a word addressed array that supports loads and stores. Stores apply the byte enable mask so that partial word writes update only the selected bytes, which is essential for correct execution of real software. For this initial phase, wait behavior was simplified by responding immediately with no inserted stalls. This kept the environment deterministic and made it easier to distinguish functional issues from timing effects.

4.1.2 Toolchain and Environment Debugging

After the basic environment was assembled, the main engineering effort was to debug the testbench itself. Early failures were dominated by environment errors rather than processor logic. The first major class of issues came from driving signals with incorrect direction or incorrect procedural semantics. The simulator enforces strict rules about single driver behavior for variables and about not assigning to nets that are not owned by the testbench. Errors of this type typically indicate that the testbench is accidentally assigning to signals that should be owned by the processor, or that the same variable is being assigned in more than one process. The resolution was to ensure that every signal driven by the testbench was declared as a testbench output and assigned from a single procedural source, while processor driven signals were treated strictly as read only inputs. Memory update logic was also structured carefully so that the simulator could not interpret the design as having multiple drivers for the same storage.

Once compilation issues were resolved, the next challenge was a simulation that appeared inactive, with no meaningful progress in the program counter and with undefined or repetitive instruction values. Two root causes can produce this symptom. Either the processor was never released into execution mode, or the instruction memory was not correctly populated. The project design specifies that the processor requires a start command through the control connection. The testbench therefore issues a single control write of a defined start value after reset is released. After verifying that the start command was applied, the remaining blocker was instruction loading.

The most consequential fix in this phase was handling program images stored as raw binary files. An initial approach attempted to load the program using a text based memory loading system task, which expects ASCII representations of values. Feeding it a raw binary file results in illegal character parsing, and the memory ends up containing incorrect values that look like repeated no operation instructions. This manifested as the processor repeatedly seeing the canonical no operation encoding and failing to execute meaningful code. The correct solution was to read the program file as a binary stream, extract four bytes per instruction word, and assemble each word in little endian order to match the processor memory layout. After adopting binary file reading and proper byte ordering, the instruction fetch path began returning valid instruction words, and the debug instruction output confirmed the same values, indicating that the processor was now executing the intended program. During this update, a toolchain constraint also had to be handled. The simulator version in use required that all variable declarations appear before any procedural statements inside an initialization block, which required moving file related variables to an appropriate scope. This was a syntactic requirement rather than a design change, but resolving it was necessary for a stable build.

A further runtime issue arose from a foreign function mechanism used by the original VHDL debug infrastructure. The processor design included several debug and IO helper subprograms intended to call external C routines through a foreign language interface. In the given environment, no external shared library was linked,

and when the processor attempted to invoke these hooks, the simulation aborted with a fatal null pointer error. The project constraints ruled out adding C code and also discouraged invasive changes to the processor. The adopted solution was to disable foreign binding and provide pure HDL stub implementations for those routines. Procedures were implemented as no operation stubs, while functions returned safe defaults, which allowed the simulation to proceed. This change did not alter the core processor pipeline behavior for instruction fetch, execute, memory access, and register writeback. It only removed optional external side effects such as console printing and external byte level IO modeling. After modifying the debug package, a standard VHDL dependency issue appeared. When a package changes, all dependent compilation units become out of date. The fix was to recompile the package first and then recompile all dependent VHDL entities in correct order before elaborating the top level design again, which restored successful binding.

With these issues resolved, the initial testbench achieved its intended outcome. It could load a raw binary program into an instruction memory model, start the processor deterministically through the control command, provide instruction and data memory behavior consistent with the processor interfaces including correct byte masked stores, and capture execution relevant debug information into a structured log. The logging policy recorded only register writeback events, so the log contained fewer entries than the total number of executed instructions. This was expected because many instructions do not write a register, including branches, comparisons, stores, and various control flow operations. Waveform inspection confirmed that the program counter advanced and that the instruction stream evolved as expected, indicating that the processor was executing normally and that the reduced log density reflected the chosen trigger condition rather than a lack of progress. Overall, this baseline phase produced a stable simulation infrastructure that later enabled differential testing and hardware deployment by providing deterministic memories, reliable startup, and a debug trace format that could be reused across later verification stages.

4.2. Differential Verification Testbench

The second milestone focused on functional correctness verification by comparing the processor execution against a trusted reference model.

4.2.1 Differential Verification Setup and Trace Alignment

Rather than introducing an online checker inside the simulation environment, the verification flow relied on generating two execution traces in the same format and then comparing them offline. This approach kept the simulation environment relatively simple while still enabling precise localization of functional mismatches. A trimmed version of the CoreMark benchmark was selected as the primary test program because it provides a realistic mixture of integer operations, control flow,

and memory accesses, while remaining small enough to run efficiently in simulation. CoreMark is widely used as a portable embedded benchmark and its workload includes byte and halfword memory operations and pointer based data structures, which are particularly effective at exposing subtle data path and memory access errors.

The reference trace was produced using Spike, an instruction set architecture level simulator for RISCV. Spike executes the program according to the architectural specification and can emit a deterministic record of key commit events. The trace format used in this project recorded four fields per event: the program counter value, the instruction word, the destination register index for a register write, and the value written to that register. This format aligns well with the processor debug outputs already available in simulation, since the debug interface exposes the current program counter and instruction word along with explicit register writeback information. The simulation trace was produced by running the processor in ModelSim with the existing debug driven logging mechanism, so that every time the processor committed a register writeback, the testbench emitted a corresponding line in the same four field format. The resulting two logs were then compared offline. Even though the logs do not contain every executed instruction, since many instructions do not write back to a register, the writeback triggered trace is still sufficient for differential verification because any divergence in the architectural state will typically appear as a different register destination or a different written value at some writeback point. When a mismatch occurs, the corresponding program counter and instruction word provide an anchor for debugging and for identifying the failing instruction class.

4.2.2 Mismatch Symptoms and Root Cause Localization

Using this differential method, a consistent mismatch was observed during CoreMark execution. The symptoms were characteristic of incorrect handling of sub word loads. In the trace, a load instruction that should have written a small positive value into a register instead wrote a value whose high bits were unexpectedly nonzero. In a typical manifestation, the reference trace would contain a register writeback of the form where the written value had the expected low byte or low halfword, while all upper bits were zero for an unsigned load. The simulation trace, by contrast, showed the same destination register written with a value that preserved unrelated high order bits, effectively contaminating the result. A second and more discriminating symptom appeared for halfword loads at addresses with a two byte offset within a 32 bit word. In those cases, the reference trace showed a value derived from the upper half of the fetched word, while the simulation trace showed a value derived from the lower half, indicating that the address dependent selection of the correct halfword was incorrect. In an offline comparison workflow, these mismatches are easy to isolate because the first offending line identifies both the failing instruction and the architectural state transition that differs, allowing the developer to focus immediately on the load data path rather than on unrelated pipeline control.

The root cause was located in the memory stage logic responsible for formatting

load results before they are forwarded to the writeback stage. The design already included an intermediate aligned read value that effectively shifted the 32-bit read data according to the address low bits so that a target byte or halfword would be positioned in the least significant bits. However, the original implementation did not consistently apply truncation and extension rules required by the RISC V specification. For unsigned byte and unsigned halfword loads, the result must be zero extended, meaning only the least significant 8 or 16 bits of the aligned value are preserved and all upper bits are cleared. If the implementation instead forwards the entire aligned 32 bit value without truncation, any residual data in upper bits will incorrectly propagate into the destination register. For signed loads, the result must be sign extended based on the most significant bit of the selected byte or halfword. The original implementation also contained a subtle addressing error for signed halfword loads, where the low 16 bits were taken from the unshifted raw read data rather than from the aligned intermediate value. This mistake produces correct results when the halfword address offset is zero but fails when the offset is two bytes, which matches the mismatch pattern observed in the traces.

4.2.3 Fix for Sub Word Loads and Trace Consistency

The corrected version fixed both classes of problems by enforcing explicit truncation for byte and halfword loads, applying zero extension for unsigned loads, applying sign extension based on the selected field for signed loads, and ensuring that the selected 16 bit payload for halfword loads is derived from the aligned intermediate value rather than from the unaligned raw bus data.

In addition to correcting the load data formatting, the updated implementation improved trace reliability by ensuring that the debug program counter and instruction word associated with a writeback event were consistently forwarded along the pipeline path that produces the writeback record. This change does not alter architectural execution, but it ensures that the trace line printed for a given register writeback is tagged with the correct instruction context, which is essential for differential debugging. With these fixes in place, the simulation writeback trace became consistent with the Spike reference trace for CoreMark, and the previously observed mismatches for byte and halfword loads disappeared. This phase therefore demonstrated the value of an offline differential methodology.

This testbench required no intrusive changes to the simulation environment, yet it provided a clear and reproducible mechanism to detect a functional deviation, localize it to a specific instruction class, and validate the correctness of the subsequent fix through trace convergence.

4.3. Synthesizable Testbench

The third milestone transitioned the project from a simulation oriented environment into a hardware deployable test platform that could run on a FPGA board. The primary objective of this phase was to perform a synthesis driven refactoring while

preserving the same external bus interface as the previous testbench version. In particular, the platform continued to interact with the processor through the Avalon style instruction fetch, data access, and control connections that the processor itself uses. Maintaining this interface was important because it ensured functional continuity between simulation and FPGA deployment. It also ensured that the testbench remained a realistic platform model rather than a simulator specific harness.

4.3.1 Synthesizable Platform Refactoring and IP Based Memories

A central part of this refactoring was the conversion from a debug and logging driven testbench into a bus oriented platform model. In simulation, the environment relied heavily on file input and output, dynamic memory arrays, and runtime trace printing. These features are powerful for verification but are not suitable for synthesis. In the FPGA oriented version, logging was removed and the dependency on dedicated debug outputs was eliminated so that the environment could be expressed purely through the same instruction fetch, data access, and control connections used by the processor during normal execution. This interface discipline improved maintainability and created a clean boundary where the processor could be integrated with a hardware platform without introducing simulator specific constructs.

A functional change in this phase was the replacement of behavioral instruction and data memories with vendor supported memory IP blocks. In simulation, both instruction storage and data storage were represented as word indexed arrays, and instruction content was populated through raw binary file reading. On FPGA, the instruction memory was implemented using a single port read only memory component generated through the Quartus IP catalog. This ROM was configured to deliver 32 bit words and to be initialized at configuration time from a memory initialization file. The data memory was implemented using a single port random access memory component, also generated through the IP catalog. With this change, memory behavior was no longer inferred from procedural logic but mapped explicitly onto dedicated on chip memory resources, which improves both predictability and synthesis robustness.

Program initialization became an explicit part of the hardware build flow. The selected benchmark program image was prepared in a memory initialization format suitable for the ROM IP. During development, several pitfalls were encountered related to file format interpretation and tool strictness. The memory initialization mechanism distinguishes between word addressed and byte addressed interpretations, and incorrect assumptions about this mapping can cause warnings or incorrect program content at runtime. In addition, the initialization file syntax is rigid and requires a complete header and proper termination. The practical resolution was to adopt a strictly valid initialization file structure with correct width declarations, explicit radix declarations, a content section, and a required end marker. After the initialization file was accepted by the toolchain, the ROM content was reliably embedded into the FPGA configuration so that the processor could fetch real instructions immediately after startup without any runtime file loading.

4.3.2 Seven Segment Instruction Count Display

To provide a hardware visible indication of execution progress, a seven segment display subsystem was added as a simple output peripheral. This subsystem consisted of a digit encoding module that maps numerical values to segment patterns, and a higher level display driver that maintains an internal instruction counter. Since the FPGA oriented build removed reliance on processor debug ports, the counter was driven by external bus level activity. Each successful instruction fetch event was treated as one executed instruction for the purpose of progress visualization, and the accumulated count was periodically sampled and presented on the two digit display. The display update rate was deliberately reduced so that changes remained visible to a human observer, and the displayed value was derived from the large counter in a scaled form to accommodate long running programs. This peripheral therefore served as a practical on board proof of life. It demonstrated that the processor was continuously fetching and executing code, and it provided a stable demonstration output without requiring serial consoles or file based tracing.

Another important part of the single board deployment was the systematic handling of timing and pin constraints. A dedicated timing constraint file was used to declare the board clock frequency, ensuring that the timing analyzer and fitter evaluated the design against a realistic clock requirement. The physical pin assignments were captured in the Quartus settings file so that all top level ports, including clock, reset, and the added display signals, were mapped to the correct board connectors and electrical standards. During this process, unused predefined assignments were removed to avoid conflicts and to keep the constraint set minimal and consistent with the actual top level interface. The result was a clean constraint environment where compilation could proceed from analysis through fitting and assembly reproducibly.

In summary, the single board testbench iteration achieved a complete synthesizable platform suitable for FPGA deployment while preserving the Avalon style interface used in the simulation versions. The environment was refactored to remove simulation only constructs and to rely solely on the processor external bus connections. Behavioral memories were replaced by Quartus generated ROM and RAM IP blocks with deterministic preloading through a validated initialization file. A seven segment display subsystem was integrated to show an instruction execution count and to provide an on board demonstration of continuous execution. With these elements in place, this version of the testbench could be integrated with the processor on the same FPGA board and was verified to run correctly in hardware, establishing a functionally correct single board platform as the foundation for the subsequent dual board work.

4.4. Duel-board Testbench

The fourth milestone completed the transition from a single board demonstration into the final dual board setup. The purpose of this phase was to keep the FPGA

oriented, synthesizable platform developed in the previous milestone and to move the processor onto a separate board. The processor board was responsible for instruction execution, while the testbench board provided the instruction memory and data memory services. The two boards communicated through a compact bus serial link, so that all instruction fetches and data accesses initiated by the processor could be transported across the link and reconstructed as equivalent memory transactions on the testbench board. A key requirement was to stay within the project constraint of fewer than twelve inter board pins, and to preserve the overall platform behavior already validated on the single board system.

4.4.1 Interface Transform Module

To realize the dual board partition, two VHDL modules were introduced on the testbench board to implement the bus serial protocol. The first module provided shared type definitions and constants used by the protocol state machine, ensuring consistent widths and packet structure across the design. The second module implemented the protocol itself as a finite state machine that receives two bit symbols and reconstructs complete transactions including the target address, read or write intent, and write data payload. This protocol implementation was intentionally reused from the processor side work to guarantee that both boards interpreted the on wire format identically. At a high level, the protocol performs framing, accumulation of transmitted symbols into a full word, and basic integrity checking through a parity style mechanism. Once a transaction is accepted, the module asserts a slave active response and returns two bit symbols back to the master as the read data stream.

An important functional addition made during this phase was the explicit introduction of a transaction valid marker. This marker indicates that a complete and integrity checked transaction has been received and is ready to be serviced. The motivation was practical and system level rather than theoretical. Without an explicit valid indication, an idle bus can be misinterpreted as repeated read intent, which can cause the testbench board to issue unintended memory accesses and produce false activity. By generating a valid marker only when a complete and correct frame is decoded, the downstream logic is able to distinguish true requests from idle conditions, improving robustness during bring up and preventing spurious instruction fetch counting when the boards are disconnected.

A new SystemVerilog bridge module was added on the testbench board to translate decoded bus serial transactions into the Avalon style instruction and data interfaces already used in the single board platform. This bridge takes the decoded address and the read or write intent, then drives an instruction fetch transaction or a data access transaction toward the existing memory subsystem. In this way, the core of the testbench platform remained unchanged. The instruction memory and data memory IP blocks, the program preloading mechanism, and the control style startup behavior remained consistent with the previous milestone. The only structural difference was that, instead of being driven directly by a local processor instance, the Avalon style interface was now driven by the reconstructed transactions originating from the remote processor board. The instruction count display subsystem intro-

duced earlier was also preserved. The counter continued to use the instruction fetch completion events as its progress source, but it was now gated by the transaction valid marker so that counting reflects only real, decoded activity across the link.

4.4.2 IO Design

A major engineering task in this phase was establishing a clean and reproducible pin constraint setup for both boards. The pin mapping on the testbench board was derived directly from the processor board side definitions used in the bus serial implementation, so that the two boards could be connected one to one without ambiguity. The inter board connection stayed within the pin budget by using a compact set of signals carried on the general purpose header. The link used two enable related control lines that indicate link readiness and activity, a two bit transmit data bus, a two bit address symbol bus, a single slave active response line, and a two bit receive data bus for returning read data. In addition, the clock was forwarded from the processor board to the testbench board. This produced a total of ten inter board signals, which satisfied the requirement of fewer than twelve pins while still supporting full instruction fetch and data access functionality.

Forwarding the clock from the processor board was important for correctness and simplicity. The bus serial protocol state machine advances on clock edges and interprets each two bit symbol per cycle. If the two boards were clocked independently, then symbol sampling would not be reliable and the reconstructed transactions would not match what was transmitted. By distributing the clock from the processor board, the testbench board sampled and processed the bus serial symbols in the same timing domain, eliminating the need for complex cross clock synchronization at the protocol boundary and enabling deterministic transaction decoding.

4.4.3 Dual Board Bring Up

After integrating the protocol modules, the Avalon reconstruction bridge, the shared pin constraints, and the forwarded clock, the system was brought up as a full dual board test. The observable success criterion was that the testbench board would service real instruction fetches and data accesses coming from the processor board, and that the instruction count display would progress in a stable manner when the boards were connected. The transaction valid mechanism ensured that the testbench remained quiescent when the link was idle and that visible progress corresponded to real decoded requests. With the dual board system operating, the measured instruction execution speed was 100198 instructions per second. This value was consistent with expectations for a design in which every instruction fetch and many data accesses traverse a low bandwidth two bit serial link and incur protocol overhead, and it served as a practical confirmation that the end to end architecture was functioning correctly in the final dual board configuration.

In conclusion, this dual board iteration represents the final version of the testbench

platform. It fulfills the original project goals and constraints by providing a compact, synthesizable execution environment with a minimal inter board pin budget, a well defined bus serial communication layer, and a clear on board progress indicator. The resulting design is directly applicable as a reusable test platform for both the RISCV processor and the BRISC implementation, and it forms a complete and project compliant basis for the final hardware demonstrations.

5. FPGA Communication

5.1. TB Serialization - Bit Serial Interface

One of the most critical constraints of the project is the limited number of I/O pads that are available for communication between the two FPGAs. Indeed, only twelve pins are available because of the high-temperature test setup used at KTH that can't take more I/Os. As a consequence, conventional parallel buses for instruction loading, data transfer, or debugging are not feasible. To overcome this limitation, there is no other choice than using bit-serial communication. As a result, an interface is needed to adapt and convert parallel communication into bit-serial communication.

How does a bit-serial interface work? The idea is to serialize multi-bit data words and transmit them bit by bit over a small number of physical signals. As we can imagine, it takes much more time for the data to be transmitted, and it introduces additional complexity in terms of timing, synchronization, and control logic. However, this approach significantly reduces the number of required I/O pins and that is what we want. This loss of performance is acceptable in this project because performance is not the primary objective. Indeed, we prioritize the correctness of the system and compatibility with our constraints (for instance extreme-environment constraint).

In our project, we use a master-slave communication that we will talk about in more detail later in the report. FPGA2 (the one with the TB) is responsible for generating serialized instruction streams, data inputs, and control signals, which are transmitted to the processor on FPGA1. Conversely, FPGA1 returns serialized response data and status information back to the testbench. The bit-serial interface will be placed in the FPGA2 and will need to convert the avalon I/Os of the TB into the bit serial I/Os. We can see in the picture above the minimal set of signals on the bit-serial side and the parallel avalon communication on the TB side.

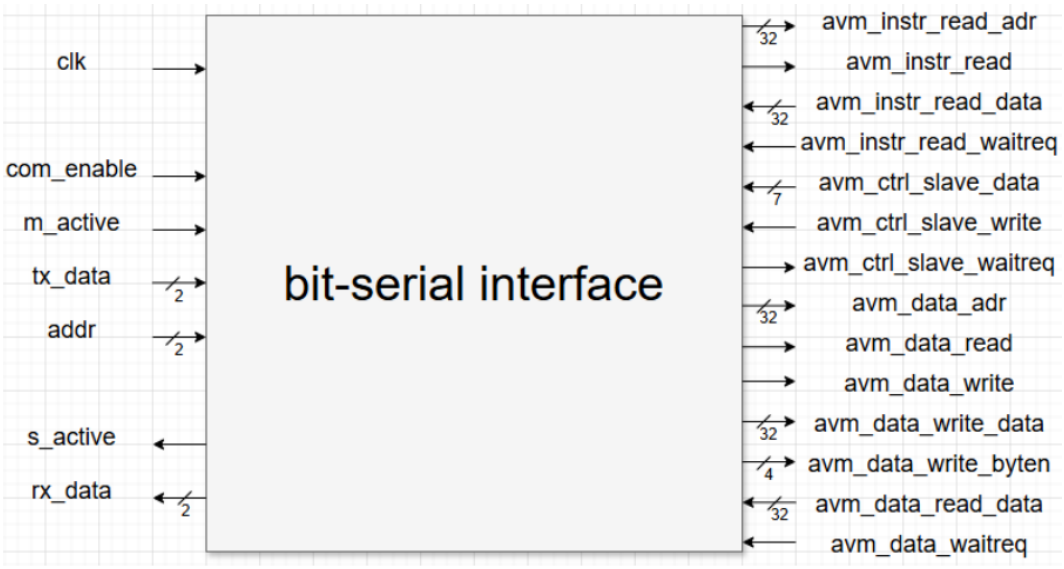

Figure 5: bit-serial interface architecture v1

We tried at first to describe this module in one block but it rapidly became a complex module, so the decision was made to describe it in two parts. The first one is bs_interface and deals with the serialisation part. The second one is avm_interface and concentrates on avalon communication.

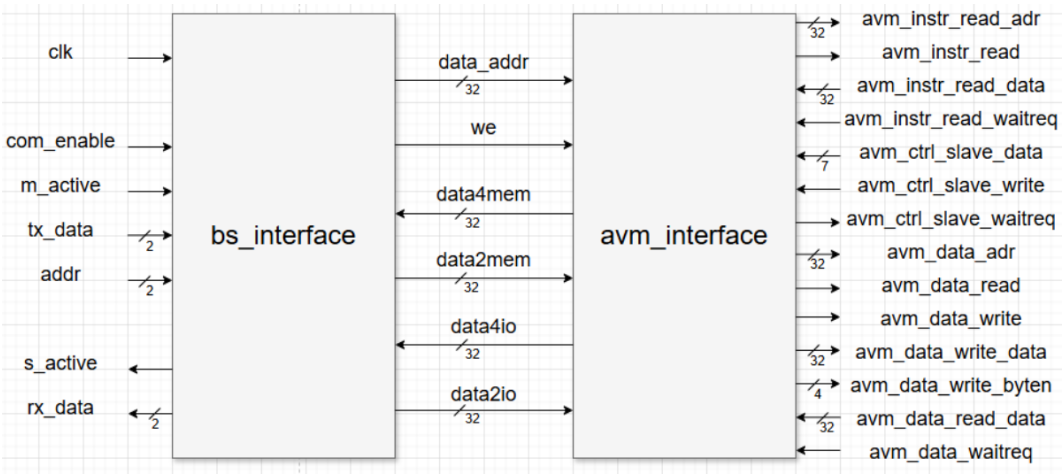

Figure 6: bit-serial interface architecture v2

5.1.1 bs_interface

As we said, the `bs_interface` module is responsible for converting parallel transactions into a bit-serial communication protocol. In order to do this, we implement it as a synchronous finite state machine (FSM).

Interface and I/O description On the serial side, the interface is connected to the CPU. In this context, it is the CPU that initiates transactions by asserting the `m_active` signal. The `tx_data` and `addr` signals carry two-bit wide serial data and address fragments from the CPU to the testbench. In response, the testbench asserts the `s_active` signal to acknowledge activity and transmits serialized data back to the processor through the `rx_data` signal.

On the parallel side, the signals `data_addr` and `we` control memory addressing and write enable, `data2mem` and `data2io` write data while `data4mem` and `data4io` read data.

Finite state machine architecture The `bs_interface` is implemented as a finite state machine. The FSM ensures that each phase of communication is executed in a well-defined order. The main states of the FSM are:

- **IDLE**: Waiting state where no communication is active.
- **WR_TX**: Serialization and transmission of write data and address.
- **WR_WAIT_ACK / WR_WAIT_NACK**: Waiting for acknowledgment or rejection of a write transaction.
- **RD_TX_ADDR**: Serialization and transmission of the read address.
- **RD_WAIT_ACK / RD_WAIT_NACK**: Validation of the received address.
- **RD_TX_DATA**: Serialization and transmission of read data back to the master.

And it follows the graph logic.

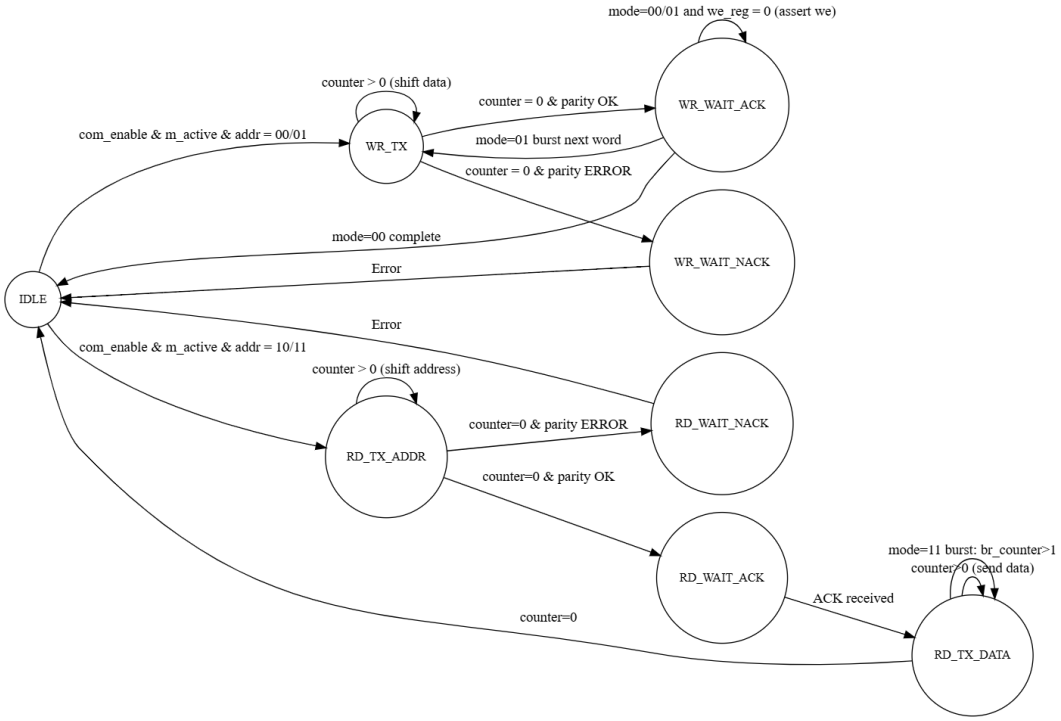

Figure 7: serialisation FSM graph

Module logic A transaction starts when both `com_enable` and `m_active` are asserted. It determines the operation type (read, write) and organizes the serialization or deserialization of addresses and data accordingly. Serialization is implemented using a counter-driven mechanism that transmits or receives two bits per clock cycle. It can't go in the next state until the counter is not 0. The module proceeds also to a parity check that can detect invalid transactions and stop them (you can see on the graph the 2 states read or write wait_nack). Then, the write part enables the data to be forwarded to either memory or I/O depending on the target address. And the read part enables the data to be fetched from memory or I/O.

5.1.2 avm_interface

The `avm_interface` module acts as an adapter between the parallel signals produced by the `bs_interface` and the Avalon memory-mapped interfaces used inside the testbench. There is no serialisation here since the communication is parallel to parallel. Therefore this module focuses on routing and translating simple read and write requests into valid Avalon transactions.

Interface and I/O description On the bs_interface side, we have a simple parallel interface. The module receives an address (`data_addr`), a write enable signal (`we`) and write data (`data2mem`). It outputs read data through `data4mem` or `data4io`, depending on the accessed address.

On the other side of the module, it uses an Avalon memory-mapped (Avalon-MM) protocol. It provides a standardized and modular interface for instruction fetches, data accesses, and control signaling. In this project, the Avalon interface is used in a simplified way. Indeed, the testbench always responds immediately to read and write requests, it simplifies some signals. Instruction accesses are handled through a read-only port while data accesses support both reads and writes. And a control slave interface is used to transmit simple commands such as starting program execution.

Module logic The `avm_interface` implements consist of a simple mapping between both sides. The instruction fetch path is permanently enabled, allowing continuous instruction reads. For data accesses, the module asserts either a read or write operation depending on the `we` signal. Write data is forwarded directly to the Avalon data port, while read data returned by the Avalon interface is captured on the rising edge of the clock and routed to either the memory or I/O output, based on the most significant address bit.

By avoiding internal state machines and wait-state management, the module `avm_interface` remains simple, deterministic, and easy to analyze. This design choice is consistent with the project's objectives, as all protocol complexity is intentionally concentrated in the `bs_interface`, leaving the Avalon adaptation layer minimal and robust.

6. Conclusion

6.1. Conclusion

6.2. Future Work

7. References

References

[1] KTH Royal Institute of Technology, Official Website. https://www.kth.se

[2] Intel, "Intel FPGA Development Boards," https://www.altera.com/downloads

[3] RISC-V Foundation, "RISC-V Instruction Set Manual," https://docs.riscv.org/reference/isa/v20240411/unpriv/rv32.html

[4] Terasic, "DE0-Nano-SoC User Manual," https://www.terasic.com.tw/attachment/archive/941/DE0-Nano-SoC_User_manual_rev.D0.pdf

[5] Farnell, "7-Segment Display Datasheet," https://www.farnell.com/datasheets/2863910.pdf